

CS 802 OBJECT ORIENTED SOFTWARE ENGINEERING

1. OO Principles

1.1 Definition of Object Oriented Principles

Object Oriented Principles are the basic rules and concepts used in Object Oriented Programming (OOP) and Object Oriented Software Engineering (OOSE) to design software systems efficiently.

These principles help in:

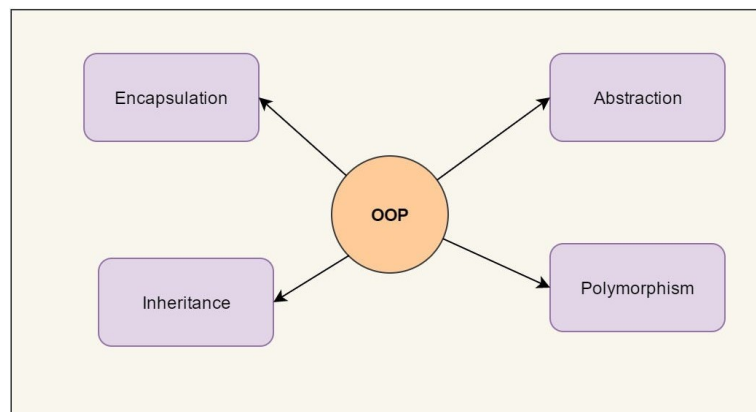
- Reusability
- Security
- Flexibility
- Easy maintenance
- Better software design

The main OO principles are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

These principles together form the foundation of object oriented systems.

1.2 Diagram of OO Principles



1.3 Encapsulation

Definition: Encapsulation is the process of wrapping data and methods together into a single unit called a class.

It protects the internal data of an object from unauthorized access.

Data is usually hidden using:

- private access modifier
- getter/setter methods

Working of Encapsulation

- Data members are declared private.
- Methods are used to access and modify data.
- Direct access from outside the class is restricted.

Example

```
class Student{
    private int marks;

    public void setMarks(int m){
        marks = m;
    }

    public int getMarks(){
        return marks;
    }
}
```

Here, marks cannot be accessed directly.

Advantages of Encapsulation

- a) Provides data security
- b) Prevents unauthorized access
- c) Improves maintainability
- d) Makes code modular
- e) Easy debugging and testing

Applications of Encapsulation

- Banking systems
- Student management systems

- Employee management software
 - Secure applications
-

1.4 Abstraction

Definition: Abstraction means hiding unnecessary implementation details and showing only important features to the user.

The user only knows what an object does, not how it works internally.

Abstraction is achieved using:

- Abstract classes
- Interfaces

Working of Abstraction

- Important functions are exposed.
- Internal logic remains hidden.
- User interacts only with essential features.

Example

When we drive a car:

- We use steering, brake, accelerator.
- We do not need to know engine internals.

Advantages of Abstraction

- a) Reduces complexity
- b) Improves security
- c) Easy to use system
- d) Increases maintainability
- e) Hides sensitive implementation details

Applications of Abstraction

- ATM machines
 - Mobile phones
 - Online payment systems
 - Operating systems
-

1.5 Inheritance

Definition: Inheritance is the process in which one class acquires properties and methods of another class.

The existing class is called:

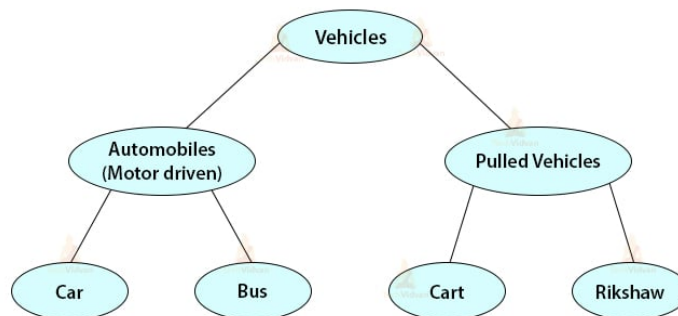
- Parent/Base/Super class

The new class is called:

- Child/Derived/Sub class

Inheritance supports code reusability.

Diagram of Inheritance



Working of Inheritance

- Child class inherits members from parent class.
- Child class can use or extend parent functionality.

Example

```
class Animal{
    void sound(){
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal{
}
```

Dog class inherits the `sound()` method.

Advantages of Inheritance

- a) Code reusability
- b) Reduces redundancy
- c) Easy maintenance
- d) Improves software extensibility
- e) Supports hierarchical classification

Applications of Inheritance

- Vehicle management systems
 - Banking applications
 - Employee hierarchy systems
 - Educational software
-

1.6 Polymorphism

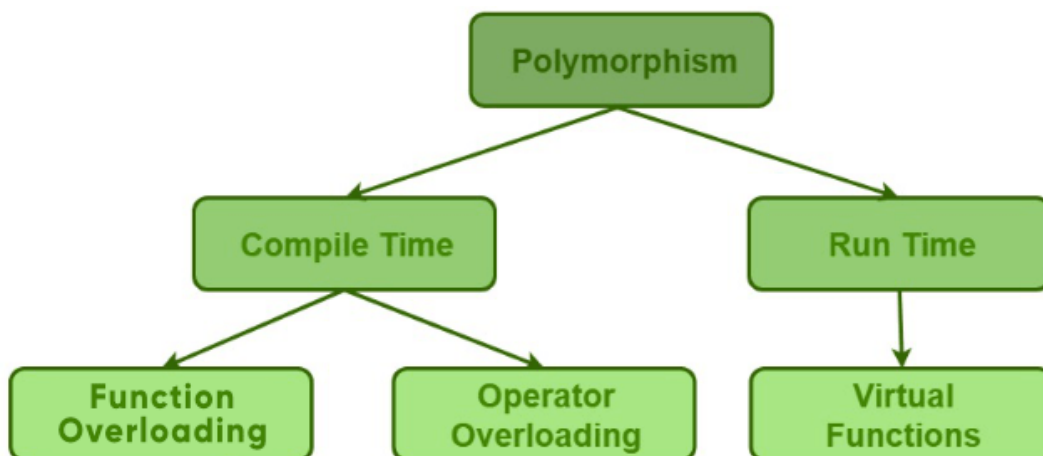
Definition: Polymorphism means “many forms”.

It allows one method or object to behave differently in different situations.

Types of polymorphism:

1. Compile Time Polymorphism
 - Method Overloading
2. Run Time Polymorphism
 - Method Overriding

Diagram of Polymorphism



Working of Polymorphism

Method Overloading: Same method name with different parameters.

Method Overriding: Child class provides different implementation of parent method.

Advantages of Polymorphism

- a) Increases flexibility
- b) Improves code reusability
- c) Easy scalability
- d) Reduces complexity
- e) Supports dynamic behavior

Applications of Polymorphism

- Graphics software
- Gaming applications
- Payment systems
- Compiler design

1.7 Comparison of OO Principles

Principle	Main Purpose	Key Feature
Encapsulation	Data Hiding	Security
Abstraction	Hide Complexity	Simplicity
Inheritance	Reuse Code	Extensibility
Polymorphism	Multiple Behavior	Flexibility

1.8 Advantages of Object Oriented Principles

- a) Improves software quality
 - b) Reduces code duplication
 - c) Makes software reusable
 - d) Simplifies maintenance
 - e) Enhances security
 - f) Makes systems scalable and modular
-

1.9 Applications of OO Principles

OO principles are used in:

- Banking systems
 - Hospital management systems
 - E-commerce websites
 - Mobile applications
 - Real-time systems
 - Enterprise software
-

1.10 Conclusion

Object Oriented Principles are the core concepts of Object Oriented Software Engineering. These principles help developers build secure, reusable, flexible, and maintainable software systems.

Encapsulation provides security, abstraction reduces complexity, inheritance promotes code reuse, and polymorphism provides flexibility. Together, these principles improve overall software design and development efficiency.

2. Conventional vs OO Design

2.1 Definition of Conventional Design

Conventional Design is a traditional software design approach in which the system is divided mainly into functions and procedures.

It focuses on:

- Sequence of operations
- Functions
- Algorithms
- Data processing

This approach is also called:

- Procedure Oriented Design
- Function Oriented Design

Languages commonly used:

- C

- Pascal

2.2 Definition of Object Oriented (OO) Design

Object Oriented Design (OOD) is a software design approach in which the system is designed using objects and classes.

It focuses on:

- Objects
- Data
- Behavior
- Real world modeling

OOD combines:

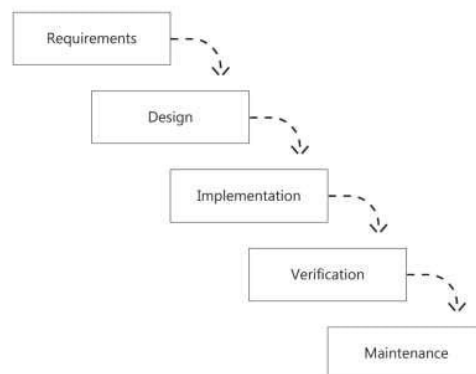
- Data
- Methods
inside a single unit called class.

Languages commonly used:

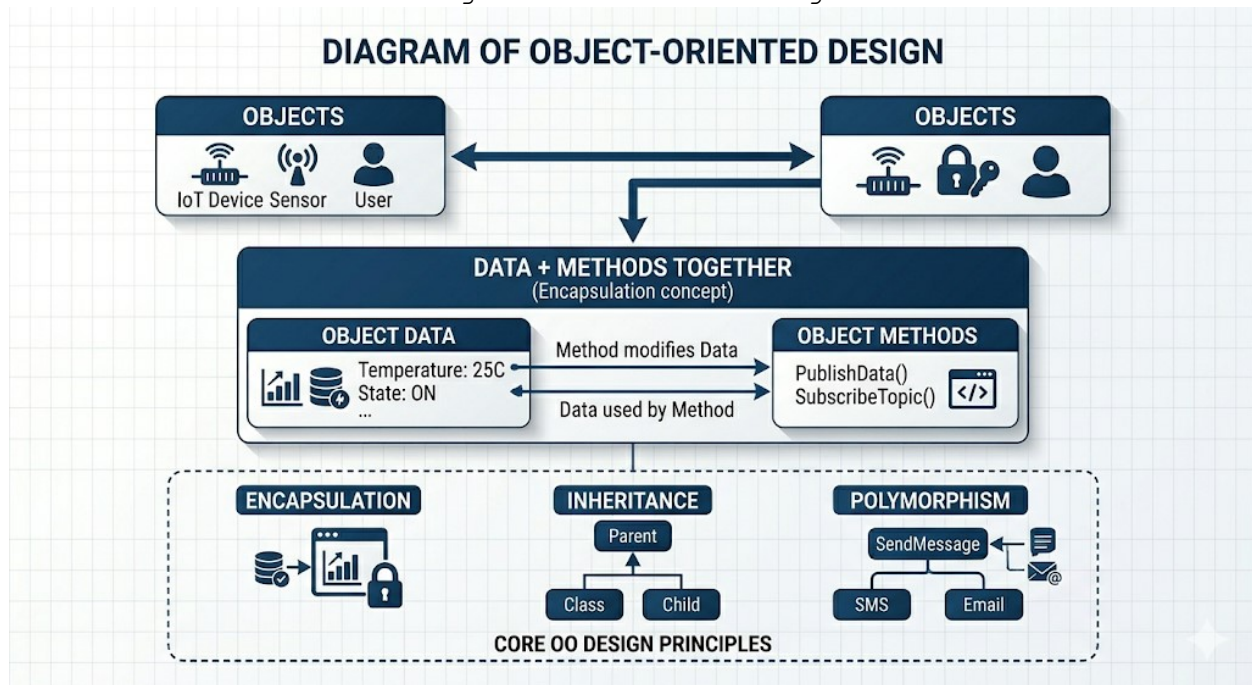
- Java
- C++
- Python
- C#

2.3 Diagram of Conventional vs OO Design

Conventional Design



Object Oriented Design



2.4 Working of Conventional Design

In conventional design:

1. System is divided into functions.
2. Functions process data separately.
3. Data is shared among functions.
4. Focus is mainly on procedures and logic.

Example

Banking system:

- Deposit()
- Withdraw()
- CalculateInterest()

Functions operate on shared data.

2.5 Working of Object Oriented Design

In OO design:

1. System is divided into objects/classes.
2. Each object contains:
 - Data
 - Methods
3. Objects communicate with each other.
4. Real-world entities are modeled as objects.

Example

Banking system:

- Account class
- Customer class
- Transaction class

Each object manages its own data and behavior.

2.6 Features of Conventional Design

- a) Function oriented
 - b) Top-down approach
 - c) Data and functions are separate
 - d) Less secure
 - e) Difficult maintenance in large systems
-

2.7 Features of OO Design

- a) Object oriented
 - b) Bottom-up approach
 - c) Data hiding using encapsulation
 - d) Reusable code
 - e) Real-world modeling
 - f) Modular structure
-

2.8 Advantages of Conventional Design

- a) Easy for small applications
- b) Simple logic implementation
- c) Faster for simple programs
- d) Less memory usage

2.9 Advantages of OO Design

- a) High code reusability
 - b) Better security through encapsulation
 - c) Easy maintenance
 - d) Better scalability
 - e) Real-world representation
 - f) Reduced code redundancy
-

2.10 Applications of Conventional Design

- Small utility programs
 - Mathematical applications
 - Simple calculators
 - Basic data processing systems
-

2.11 Applications of OO Design

- Banking systems
 - E-commerce applications
 - Hospital management systems
 - Mobile applications
 - Enterprise software
 - Real-time systems
-

2.12 Comparison Between Conventional and OO Design

Basis	Conventional Design	Object Oriented Design
Approach	Function oriented	Object oriented
Focus	Functions	Objects
Design Method	Top-down	Bottom-up
Data Handling	Shared globally	Encapsulated
Security	Less secure	More secure
Reusability	Low	High
Maintenance	Difficult	Easy

Basis	Conventional Design	Object Oriented Design
Real World Modeling	Poor	Excellent
Flexibility	Less flexible	Highly flexible
Suitable For	Small systems	Large complex systems

2.13 Limitations of Conventional Design

- a) Difficult to maintain large software
 - b) High code duplication
 - c) Weak security
 - d) Difficult debugging
 - e) Low reusability
-

2.14 Importance of OO Design

OO Design is important because:

- Modern software systems are complex.
 - It improves modularity and maintainability.
 - It supports reuse using inheritance.
 - It provides flexibility using polymorphism.
 - It models real-world systems effectively.
-

2.15 Conclusion

Conventional Design mainly focuses on functions and procedures, while Object Oriented Design focuses on objects and real-world entities.

OO Design provides better security, reusability, maintainability, and scalability compared to conventional design. Therefore, most modern software systems use Object Oriented Design approaches.

3. RUP Life Cycle

3.1 Definition of RUP

Rational Unified Process (RUP) is a software development process model developed by Rational Software Corporation.

It is:

- Object oriented
- Iterative
- Incremental
- Use-case driven
- Architecture centric

RUP provides a structured way to develop high quality software within time and budget.

3.2 Features of RUP

- a) Iterative development
 - b) Incremental software delivery
 - c) Use-case driven approach
 - d) Architecture-centric design
 - e) Risk management support
 - f) Continuous testing and integration
-

3.3 Definition of RUP Life Cycle

RUP Life Cycle is the sequence of phases used to develop software systematically.

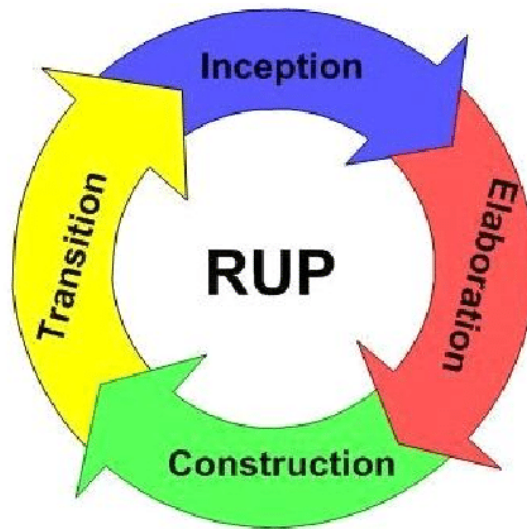
The RUP life cycle divides software development into four phases:

1. Inception Phase
2. Elaboration Phase
3. Construction Phase
4. Transition Phase

Each phase contains:

- Activities
 - Iterations
 - Testing
 - Documentation
-

3.4 Diagram of RUP Life Cycle



3.5 Working of RUP Life Cycle

The software is developed step-by-step through iterations.

Each phase performs:

- Requirement gathering
- Design
- Coding
- Testing
- Deployment

At the end of every phase, the software becomes more complete and stable.

3.6 Phases of RUP Life Cycle

3.6.1 Inception Phase

Definition: This is the initial phase of RUP where project scope and feasibility are identified.

Main Activities

- Requirement gathering
- Understanding customer needs
- Identifying project goals
- Cost estimation

- Risk identification
- Feasibility study

Output of Inception Phase

- Business case
- Initial project plan
- Basic use cases
- Scope document

Advantages

- a) Defines clear project objectives
 - b) Reduces project confusion
 - c) Identifies major risks early
-

3.6.2 Elaboration Phase

Definition: In this phase, system architecture and detailed analysis are developed.

Main Activities

- Detailed requirement analysis
- System design
- Architecture development
- Risk analysis
- Use case refinement

Output of Elaboration Phase

- Software architecture
- UML diagrams
- Detailed project plan

Advantages

- a) Reduces technical risks
 - b) Creates stable architecture
 - c) Improves project understanding
-

3.6.3 Construction Phase

Definition: In this phase, actual software coding and development are performed.

Main Activities

- Coding
- Testing
- Integration
- Database development
- UI development

Output of Construction Phase

- Working software product
- Tested modules
- User documentation

Advantages

- a) Produces executable software
 - b) Improves software functionality
 - c) Supports incremental development
-

3.6.4 Transition Phase

Definition: This is the final phase where the software is delivered to users.

Main Activities

- Software deployment
- User training
- Bug fixing
- Maintenance
- Final testing

Output of Transition Phase

- Final software product
- User manuals
- Installed system

Advantages

- a) Smooth software deployment
 - b) User satisfaction
 - c) Real-world system validation
-

3.7 Iterative Nature of RUP

RUP follows iterative development.

This means:

- Software is developed in small iterations.
- Each iteration improves the system.
- Feedback is continuously collected.

Benefits:

- Early error detection
 - Reduced project risk
 - Better quality software
-

3.8 Advantages of RUP

- a) Better project management
 - b) Early risk identification
 - c) High software quality
 - d) Continuous testing support
 - e) Flexible and scalable
 - f) Suitable for large projects
 - g) Better customer involvement
-

3.9 Applications of RUP

RUP is used in:

- Banking software
- Enterprise applications
- E-commerce systems
- Defense systems

- Healthcare systems
 - Large software industries
-

3.10 Comparison of Traditional SDLC and RUP

Basis	Traditional SDLC	RUP
Development Style	Sequential	Iterative
Risk Handling	Late	Early
Flexibility	Low	High
Customer Feedback	Limited	Continuous
Testing	At end	Continuous
Software Delivery	Single delivery	Incremental delivery

3.11 Importance of RUP

RUP is important because:

- It handles large projects efficiently.
 - It reduces software risks.
 - It improves software quality.
 - It supports object oriented development.
 - It allows continuous improvement.
-

3.12 Conclusion

RUP Life Cycle is an iterative and object oriented software development process that divides software development into four phases: Inception, Elaboration, Construction, and Transition.

It helps in managing risks, improving software quality, and developing software systematically. Due to its flexibility and structured approach, RUP is widely used in modern software engineering projects.

4. RUP Workflows

4.1 Definition of RUP Workflows

RUP Workflows are the organized activities performed during software development in the Rational Unified Process (RUP).

A workflow defines:

- What work is done
- Who performs the work
- When the work is performed
- How the work is completed

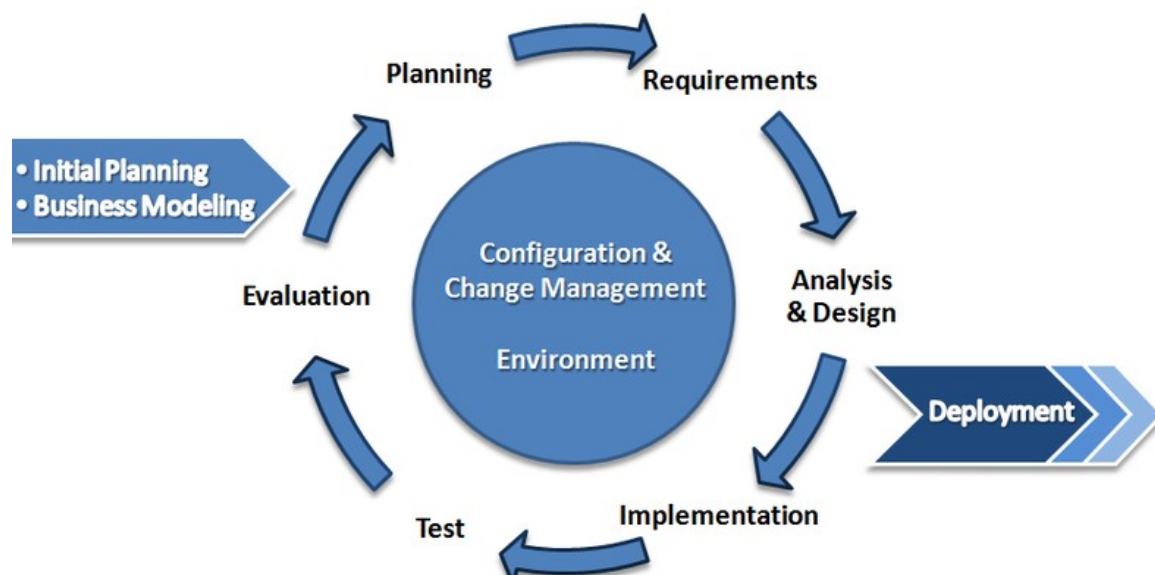
RUP workflows help in systematic and efficient software development.

4.2 Types of RUP Workflows

RUP mainly contains two categories of workflows:

1. Engineering Workflows
 2. Supporting Workflows
-

4.3 Diagram of RUP Workflows



4.4 Engineering Workflows

Engineering workflows are directly related to software development activities.

Main engineering workflows are:

1. Business Modeling
2. Requirement Workflow
3. Analysis and Design Workflow
4. Implementation Workflow
5. Testing Workflow
6. Deployment Workflow

4.5 Business Modeling Workflow

Definition: Business Modeling helps in understanding the business processes and system requirements.

It identifies:

- Business goals
- Business activities
- User requirements

Working

- Study organization structure
- Understand business operations
- Identify system users
- Prepare business models

Advantages

- a) Better understanding of system requirements
- b) Reduces requirement confusion
- c) Improves communication with clients

Applications

- Banking systems
- Hospital management
- E-commerce systems

4.6 Requirement Workflow

Definition: Requirement workflow collects and manages user requirements.

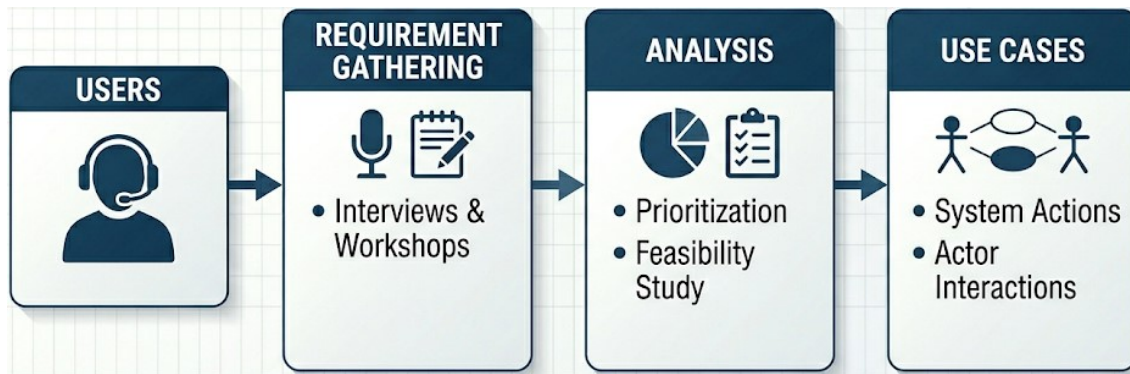
This workflow identifies:

- Functional requirements
- Non-functional requirements
- Use cases

Working

- Requirement gathering
- Requirement analysis
- Preparing use case diagrams
- Requirement documentation

Diagram



Advantages

- a) Clear understanding of system needs
- b) Reduces development errors
- c) Improves software quality

4.7 Analysis and Design Workflow

Definition: This workflow converts requirements into software design.

It defines:

- System architecture
- Classes
- Objects
- Database structure

Working

- Prepare UML diagrams
- Create class diagrams
- Design architecture
- Define object interactions

Advantages

- a) Creates proper software structure
- b) Simplifies coding process
- c) Reduces design errors

Applications

- Enterprise applications
 - Object oriented software systems
-

4.8 Implementation Workflow

Definition: Implementation workflow performs actual coding of the software system.

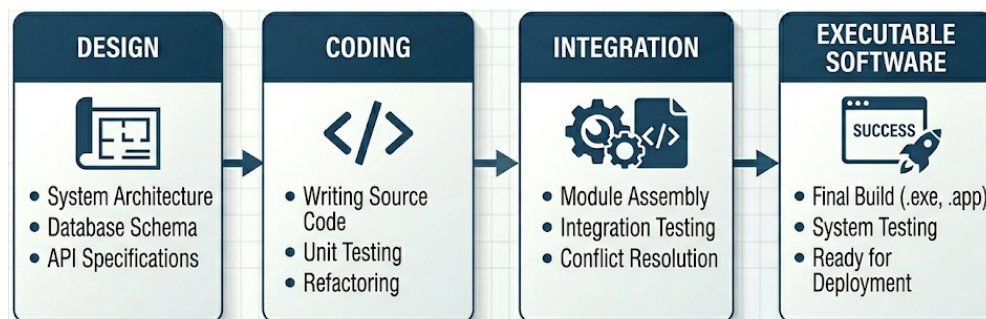
Working

- Write program code
- Integrate modules
- Build software components

Programming languages used:

- Java
- C++
- Python

Diagram



Advantages

- a) Converts design into working software
 - b) Supports modular development
 - c) Improves software functionality
-

4.9 Testing Workflow

Definition: Testing workflow checks whether the software works correctly or not.

It helps in finding:

- Errors
- Bugs
- Performance issues

Working

- Unit testing
- Integration testing
- System testing
- Validation testing

Advantages

- a) Improves software quality
- b) Detects defects early
- c) Increases reliability

Applications

- Software quality assurance
 - Security testing systems
-

4.10 Deployment Workflow

Definition: Deployment workflow delivers the software to users.

Working

- Install software
- Configure system

- Train users
- Release final product

Advantages

- a) Smooth software delivery
 - b) Better customer satisfaction
 - c) Real-world system implementation
-

4.11 Supporting Workflows

Supporting workflows help in managing and maintaining the project.

Main supporting workflows:

1. Project Management
 2. Configuration Management
 3. Environment Workflow
-

4.12 Project Management Workflow

Definition: Project management workflow manages:

- Schedule
- Cost
- Resources
- Risks

Advantages

- a) Better project control
 - b) Timely project completion
 - c) Proper resource management
-

4.13 Configuration Management Workflow

Definition: This workflow manages software versions and changes.

Working

- Track code changes
- Maintain software versions
- Manage updates

Advantages

- a) Prevents data loss
 - b) Maintains software consistency
 - c) Supports team collaboration
-

4.14 Environment Workflow

Definition: Environment workflow provides tools and infrastructure required for software development.

Examples

- IDE setup
- Testing tools
- Database tools
- UML tools

Advantages

- a) Improves development efficiency
 - b) Provides standard environment
 - c) Simplifies software development
-

4.15 Advantages of RUP Workflows

- a) Organized software development
 - b) Better project management
 - c) Continuous testing and improvement
 - d) Early risk detection
 - e) Improved software quality
 - f) Better communication among team members
-

4.16 Applications of RUP Workflows

RUP workflows are used in:

- Banking software
 - E-commerce systems
 - ERP software
 - Government projects
 - Healthcare systems
 - Large enterprise applications
-

4.17 Comparison Between Engineering and Supporting Workflows

Basis	Engineering Workflow	Supporting Workflow
Purpose	Develop software	Support development
Focus	Coding and design	Management and maintenance
Activities	Analysis, coding, testing	Planning, tracking, environment
Output	Software product	Project support

4.18 Conclusion

RUP Workflows are the structured activities used in Rational Unified Process for systematic software development.

Engineering workflows handle development activities like requirements, design, coding, testing, and deployment, while supporting workflows manage project control and environment setup.

These workflows improve software quality, reduce risks, and ensure successful project completion.

5. 4+1 View Model

5.1 Definition of 4+1 View Model

The 4+1 View Model is a software architecture model proposed by Philippe Kruchten for describing the architecture of a software system from different perspectives.

It helps developers, designers, testers, and users understand the system clearly.

The model contains:

- 4 architectural views
- +1 use case view

The views together describe the complete software system.

5.2 Purpose of 4+1 View Model

The 4+1 View Model is used to:

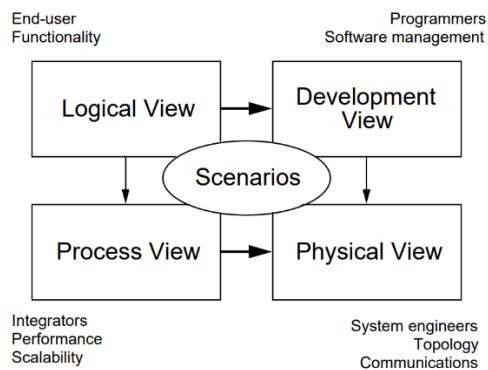
- Represent software architecture clearly
 - Handle complex systems
 - Improve communication among team members
 - Support object oriented design
 - Separate different system concerns
-

5.3 Components of 4+1 View Model

The model contains the following views:

1. Logical View
 2. Process View
 3. Development View
 4. Physical View
 5. Use Case View (+1)
-

5.4 Diagram of 4+1 View Model



5.5 Working of 4+1 View Model

Each view focuses on a different aspect of the software system.

- Logical View → Functional requirements
- Process View → Runtime behavior
- Development View → Software organization
- Physical View → Hardware deployment
- Use Case View → User requirements and scenarios

Together, all views describe the complete architecture.

5.6 Logical View

Definition: Logical View describes the functional requirements of the system.

It focuses on:

- Classes
- Objects
- Relationships
- Main functionalities

This view is mainly used by:

- Designers
- Developers

Diagram Representation

Usually represented using:

- Class diagrams
- Object diagrams

Working

- Identify classes and objects
- Define relationships
- Model system functionality

Advantages

- a) Clear understanding of system functions
- b) Supports object oriented analysis
- c) Improves software design

Applications

- Banking systems
 - Student management systems
 - Hospital systems
-

5.7 Process View

Definition: Process View describes the dynamic behavior and runtime processes of the system.

It focuses on:

- Concurrency
- Performance
- Communication
- Synchronization

This view is mainly used by:

- System engineers
- Performance analysts

Diagram Representation

Represented using:

- Activity diagrams
- Sequence diagrams

Working

- Define processes and threads
- Manage communication between processes
- Handle synchronization

Advantages

- a) Improves system performance
- b) Supports multitasking
- c) Helps in concurrency management

Applications

- Real-time systems
 - Distributed systems
 - Network applications
-

5.8 Development View

Definition: Development View describes the organization of software modules in the development environment.

It focuses on:

- Components
- Packages
- Libraries
- Source code structure

This view is mainly used by:

- Programmers
- Software managers

Diagram Representation

Represented using:

- Component diagrams
- Package diagrams

Working

- Organize software modules
- Divide software into components
- Manage development structure

Advantages

- a) Simplifies software development
- b) Improves code maintainability
- c) Supports team development

Applications

- Large enterprise projects
 - Modular applications
-

5.9 Physical View

Definition: Physical View describes the hardware deployment of the software system.

It focuses on:

- Servers
- Devices
- Networks
- Hardware nodes

This view is mainly used by:

- System administrators
- Network engineers

Diagram Representation

Represented using Deployment diagrams.

Working

- Map software to hardware
- Define network structure
- Manage deployment environment

Advantages

- a) Improves deployment planning
- b) Supports distributed systems
- c) Enhances system reliability

Applications

- Cloud systems
 - Client-server systems
 - IoT systems
-

5.10 Use Case View (+1 View)

Definition: Use Case View describes the system functionality from the user's perspective.

It contains:

- Use cases
- Actors
- User interactions

This view connects all other views.

Diagram Representation

Represented using Use case diagrams.

Working

- Identify user requirements
- Define system interactions
- Validate architecture

Advantages

- a) Improves requirement understanding
- b) Helps in testing
- c) Ensures customer satisfaction

Applications

- ATM systems
 - Online shopping systems
 - Railway reservation systems
-

5.11 Advantages of 4+1 View Model

- a) Clear architectural representation
 - b) Better communication among teams
 - c) Supports complex software systems
 - d) Improves software quality
 - e) Simplifies system understanding
 - f) Supports object oriented development
-

5.12 Applications of 4+1 View Model

The 4+1 View Model is used in:

- Enterprise applications
 - Banking systems
 - E-commerce software
 - Healthcare systems
 - Distributed systems
 - Large software projects
-

5.13 Comparison of Different Views

View	Focus Area	Main Users
Logical View	Functional design	Developers
Process View	Runtime behavior	System engineers
Development View	Software modules	Programmers
Physical View	Hardware deployment	Network engineers
Use Case View	User requirements	Customers/Testers

5.14 Importance of 4+1 View Model

The 4+1 View Model is important because:

- One diagram cannot describe complete software architecture.
 - Different stakeholders require different views.
 - It improves understanding of large systems.
 - It supports better software planning and maintenance.
-

5.15 Conclusion

The 4+1 View Model is an important architectural model used in Object Oriented Software Engineering to describe software systems from multiple perspectives.

It includes Logical, Process, Development, Physical, and Use Case views, which together provide a complete understanding of the software architecture. This model improves communication, design quality, and system organization in large software projects.

6. Use Case Diagram

6.1 Definition of Use Case Diagram

A Use Case Diagram is a behavioral diagram in UML used to represent the interaction between users (actors) and a system.

It shows:

- What the system does
- Who interacts with the system
- Different functionalities provided by the system

Use Case Diagram helps in understanding system requirements from the user's point of view.

It is mainly used during the requirement analysis phase of software development.

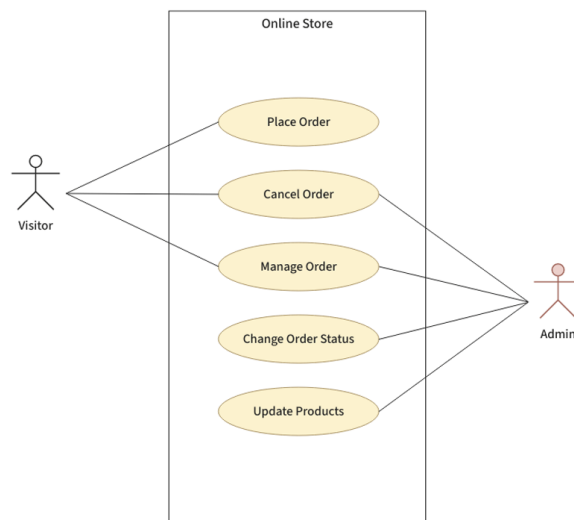
6.2 Main Components of Use Case Diagram

Component	Meaning
Actor	User or external system interacting with system
Use Case	Function or service provided by system
System Boundary	Defines scope of system
Association	Communication between actor and use case
Include Relationship	One use case includes another
Extend Relationship	Optional behavior added to use case

6.3 Basic Symbols Used in Use Case Diagram

Symbol	Purpose
Stick Figure	Actor
Oval Shape	Use Case
Rectangle	System Boundary
Line	Association
◊	Mandatory functionality
◊	Optional functionality

6.4 Diagram of Use Case Diagram



6.5 Working of Use Case Diagram

Step 1: Identify Actors

Actors are users or external systems interacting with the software.

Examples:

- Customer
- Admin
- Student
- Teacher
- Bank Server

Step 2: Identify Use Cases

Use cases represent system functionalities.

Examples in ATM:

- Withdraw Cash
- Balance Inquiry
- Deposit Money
- Change PIN

Step 3: Draw Associations

Associations show interaction between actor and system functionality.

Example: Customer → Withdraw Cash

Step 4: Define Relationships

Relationships are used to connect use cases.

i) Include Relationship

Used when one use case always includes another.

Example: Withdraw Cash includes Authentication

Withdraw Cash ----<<include>>---- Authentication

ii) Extend Relationship

Used when additional optional behavior occurs.

Example: Print Receipt extends Withdraw Cash

Print Receipt ----<<extend>>---- Withdraw Cash

Step 5: Define System Boundary

System boundary separates internal system functions from external actors.

6.6 Example of Use Case Diagram

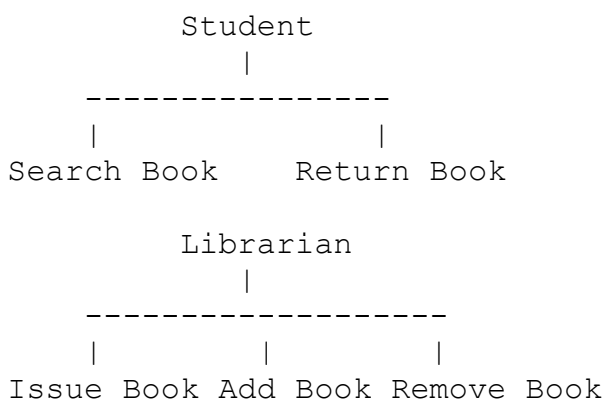
Library Management System

Actors

- Student
- Librarian

Use Cases

- Search Book
- Issue Book
- Return Book
- Add Book
- Remove Book



6.7 Advantages of Use Case Diagram

- a) **Easy to Understand:** Simple graphical representation for users and developers.
- b) **Requirement Identification:** Helps in gathering system requirements clearly.
- c) **Better Communication:** Improves communication between clients and developers.
- d) **System Scope Definition:** Clearly defines boundaries of system.
- e) **User-Centered Design:** Focuses on user interaction and needs.
- f) **Simplifies Complex Systems:** Large systems can be represented in simple form.

6.8 Applications of Use Case Diagram

- a) **Banking Systems:** ATM operations and online banking.

b) College Management System: Student registration and course management.

c) E-Commerce Applications: Shopping cart and payment systems.

d) Hospital Management: Patient registration and appointment systems.

e) Railway Reservation System: Ticket booking and cancellation.

f) Library Management System: Book issue and return operations.

6.9 Use Case vs Activity Diagram

Feature	Use Case Diagram	Activity Diagram
Purpose	Shows functionalities	Shows workflow
Focus	User interaction	Process flow
Complexity	Simple	Detailed
Representation	Actors and use cases	Activities and decisions
Usage	Requirement analysis	Process modeling

6.10 Important Terms

i) Actor: Entity interacting with system.

Example:

- Customer
- Student

ii) Use Case: Function performed by system.

Example:

- Login
- Register

iii) Flow of Events: Sequence of activities inside a use case.

Example:

1. User enters login ID
2. System validates user

3. Access granted

6.11 Conclusion

Use Case Diagram is one of the most important UML diagrams used in Object Oriented Software Engineering.

It helps developers understand:

- User requirements
- System functionality
- Interaction between users and software

It is simple, easy to understand, and widely used in software analysis and system design.

7. Activity Diagram

7.1 Definition of Activity Diagram

An Activity Diagram is a behavioral UML diagram used to represent the flow of activities or workflow of a system.

It shows:

- Sequence of activities
- Decision making
- Parallel processing
- Flow of control from one activity to another

Activity diagrams are mainly used to model:

- Business processes
- System workflows
- Algorithms
- User operations

They help developers understand how a process works step-by-step.

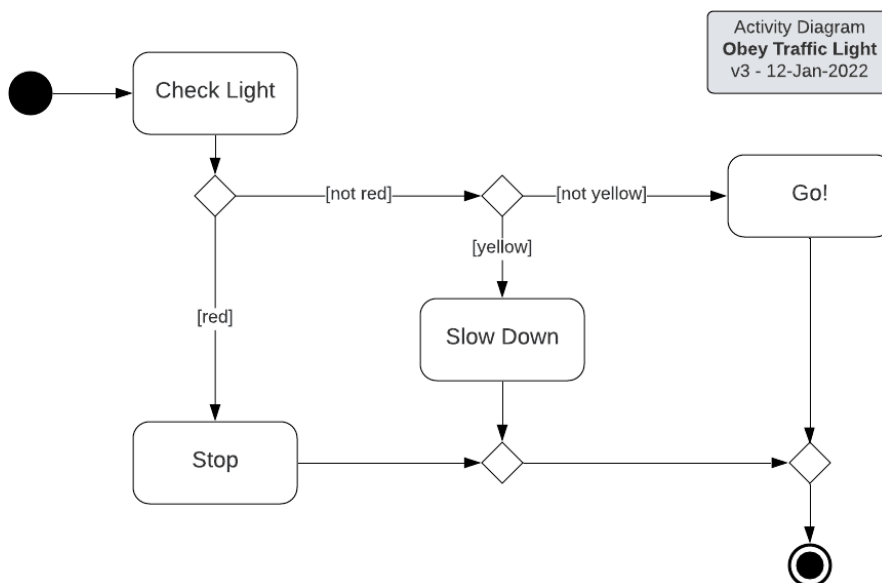
7.2 Main Components of Activity Diagram

Component	Purpose
Initial State	Starting point of process
Activity	Task or operation performed
Transition	Flow between activities
Decision Node	Condition checking
Fork	Parallel execution
Join	Combining parallel flows
Final State	End point of process

7.3 Symbols Used in Activity Diagram

Symbol	Meaning
Filled Circle	Initial State
Rounded Rectangle	Activity
Arrow	Flow/Transition
Diamond Shape	Decision
Thick Bar	Fork/Join
Double Circle	Final State

7.4 Diagram of Activity Diagram



7.5 Working of Activity Diagram

Step 1: Initial State

The process starts from the initial node.

Example: User inserts ATM card

Step 2: Activity Execution

Different activities are performed step-by-step.

Examples:

- Enter PIN
- Validate User
- Select Amount

Each activity performs a specific operation.

Step 3: Decision Making

Conditions are checked using decision nodes.

Example:

- PIN valid or invalid
- Balance sufficient or not

Based on condition, flow changes.

Step 4: Parallel Activities (Fork)

Some activities can run simultaneously.

Example:

- Sending notification
- Updating database

Fork node divides process into parallel flows.

Step 5: Join Node

Parallel activities combine again using join node.

Step 6: Final State

The process ends at the final node.

Example: Transaction completed

7.6 Advantages of Activity Diagram

- a) Easy Understanding of Workflow:** Shows complete process in simple graphical form.
 - b) Better Process Visualization:** Helps understand sequence of operations clearly.
 - c) Identifies Errors Easily:** Wrong process flow can be detected quickly.
 - d) Supports Parallel Activities:** Can model concurrent operations.
 - e) Useful for Complex Systems:** Simplifies complicated business logic.
 - f) Improves Communication:** Easy for developers, clients, and analysts to understand.
-

7.7 Applications of Activity Diagram

- a) Banking Systems:** ATM transaction workflows.
 - b) E-Commerce Systems:** Online shopping and payment process.
 - c) Hospital Management:** Patient registration and appointment systems.
 - d) Railway Reservation:** Ticket booking process.
 - e) Software Development:** Modeling algorithms and program flow.
 - f) Business Process Modeling:** Representing organizational workflows.
-

7.8 Activity Diagram vs Use Case Diagram

Feature	Activity Diagram	Use Case Diagram
Purpose	Shows workflow/process	Shows system functionality
Focus	Sequence of activities	User interaction
Complexity	More detailed	Simple overview
Main Elements	Activities and decisions	Actors and use cases
Usage	Process modeling	Requirement analysis

7.9 Important Concepts in Activity Diagram

i) Decision Node: Used for condition checking.

Example: Payment success/failure

ii) Fork Node: Used for parallel execution.

Example: Update database and send email simultaneously

iii) Join Node: Combines parallel flows into single flow.

iv) Swimlanes: Used to divide responsibilities among different users or departments.

Example:

- Customer
 - Admin
 - Payment Gateway
-

7.10 Conclusion

Activity Diagram is an important UML behavioral diagram used to represent workflows and system processes.

It helps in:

- Understanding system operations
- Modeling business processes
- Showing decision making and flow control
- Simplifying complex logic

It is widely used in software engineering for process analysis, system design, and workflow management.

8. UML Relationships

8.1 Definition of UML Relationships

UML Relationships are connections between classes, objects, and other elements in UML diagrams.

They show:

- How components interact
- Dependency between classes
- Communication among objects
- Structure of software system

Relationships are mainly used in:

- Class diagrams
- Use case diagrams
- Object-oriented design

UML relationships help in understanding the architecture and behavior of software systems.

8.2 Types of UML Relationships

There are mainly four important UML relationships:

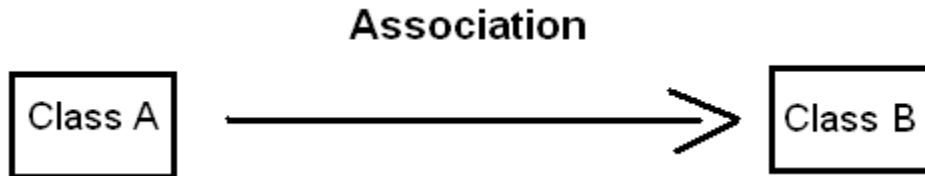
Relationship	Purpose
Association	General connection between classes
Aggregation	Weak “HAS-A” relationship
Composition	Strong “HAS-A” relationship
Inheritance (Generalization)	“IS-A” relationship
Dependency	One class depends on another

8.3 Association Relationship

Definition: Association represents a normal relationship between two classes where objects communicate with each other.

It shows that one object uses or interacts with another object.

Diagram of Association



Working

- A student enrolls in a course.
- Both classes can exist independently.

Association may be:

- One-to-one
- One-to-many
- Many-to-many

Example

- Teacher teaches Student
- Customer places Order

Advantages

- Shows communication between classes
 - Easy to understand system interaction
-

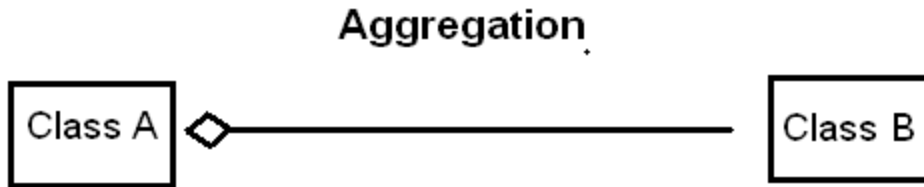
8.4 Aggregation Relationship

Definition: Aggregation is a special type of association representing a weak HAS-A relationship.

In aggregation:

- Child object can exist independently of parent object.

Diagram of Aggregation



Working

- Department contains professors.
- If department is removed, professors can still exist.

Example

- Library contains Books
- Team contains Players

Advantages

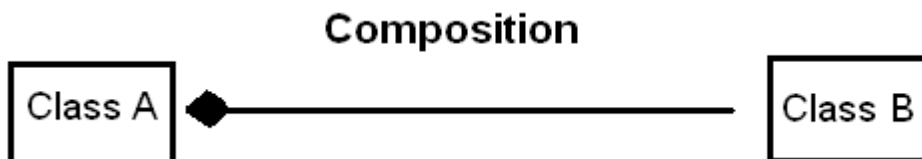
- Represents ownership clearly
 - Reusability of objects possible
-

8.5 Composition Relationship

Definition: Composition is a strong form of aggregation representing strong ownership.

In composition Child object cannot exist without parent object.

Diagram of Composition



Working

- House contains rooms.
- If house is destroyed, rooms also get destroyed.

Example

- Car contains Engine

- Human body contains Heart

Advantages

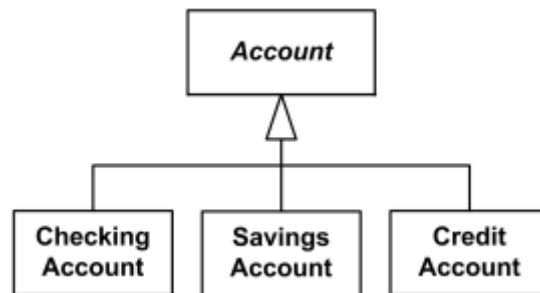
- Strong object dependency representation
 - Better lifecycle management
-

8.6 Inheritance (Generalization)

Definition: Inheritance represents IS-A relationship between classes.

A child class inherits properties and methods from parent class.

Diagram of Inheritance



Working

- Dog inherits properties of Animal.
- Reuse of code becomes possible.

Example

- Car is a Vehicle
- Student is a Person

Advantages

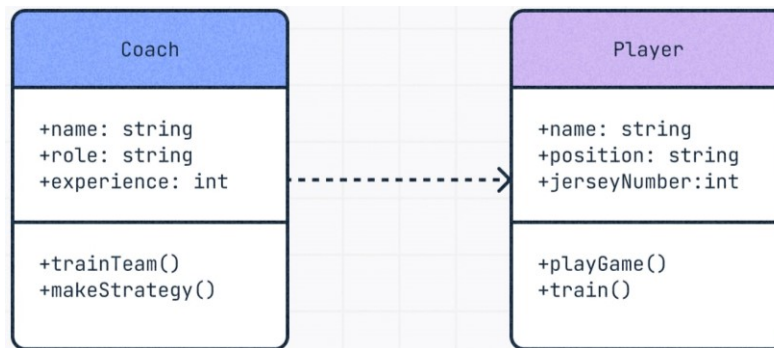
- Code reusability
 - Reduces duplication
 - Supports polymorphism
-

8.7 Dependency Relationship

Definition: Dependency shows that one class depends on another class temporarily.

Changes in one class may affect the dependent class.

Diagram of Dependency



Working

- Customer class uses PaymentService temporarily.
- Dependency exists only during operation.

Example

- Login page uses Database service
- Report generator uses Printer

Advantages

- Shows temporary usage
- Helps identify module dependency

8.8 Comparison Between UML Relationships

Feature	Association	Aggregation	Composition	Inheritance	Dependency
Relationship Type	General	Weak HAS-A	Strong HAS-A	IS-A	Uses
Ownership	No ownership	Weak ownership	Strong ownership	Parent-child	Temporary
Lifetime Dependency	No	No	Yes	No	No

Feature	Association	Aggregation	Composition	Inheritance	Dependency
Symbol	Line	Hollow diamond	Filled diamond	Arrow triangle	Dashed arrow
Example	Student-Course	Team-Player	House-Room	Animal-Dog	User-Printer

8.9 Advantages of UML Relationships

- a) **Better System Understanding:** Shows connection between system components clearly.
 - b) **Simplifies Design:** Helps developers organize classes properly.
 - c) **Improves Reusability:** Inheritance and aggregation support reuse.
 - d) **Easy Maintenance:** Dependencies between modules become clear.
 - e) **Supports Object-Oriented Design:** Represents real-world relationships effectively.
-

8.10 Applications of UML Relationships

- a) **Software Design:** Used in class diagrams during development.
 - b) **Banking Systems:** Relationship between Customer, Account, and Transaction.
 - c) **E-Commerce Applications:** Customer, Product, Order relationships.
 - d) **Hospital Management:** Doctor, Patient, Department relationships.
 - e) **Educational Systems:** Student, Teacher, Course relationships.
-

8.11 Real Life Examples of UML Relationships

Real Life Example	Relationship Type
Student enrolls in Course	Association
Team has Players	Aggregation
House has Rooms	Composition
Dog is an Animal	Inheritance

Real Life Example	Relationship Type
Customer uses Payment Service	Dependency

8.12 Conclusion

UML Relationships are important for representing connections between classes and objects in Object-Oriented Design.

They help developers:

- Understand system structure
- Design reusable software
- Represent real-world scenarios
- Improve maintainability and scalability

The major UML relationships are:

- Association
- Aggregation
- Composition
- Inheritance
- Dependency

These relationships form the foundation of UML modeling and object-oriented software engineering.

9. Stereotypes

9.1 Definition of Stereotypes

Stereotypes are special UML elements used to extend the meaning of UML diagrams.

They provide additional information about:

- Classes
- Objects
- Components
- Interfaces
- Relationships

A stereotype helps developers understand the role or behavior of an element more clearly.

In UML, stereotypes are written inside double angle brackets.

Example:

```
<<interface>>  
<<entity>>  
<<controller>>
```

Stereotypes are part of UML extension mechanisms.

9.2 Purpose of Stereotypes

Stereotypes are used to:

- Add extra meaning to UML elements
- Categorize classes and components
- Improve readability of diagrams
- Represent special behaviors or responsibilities

They help make UML diagrams more descriptive and understandable.

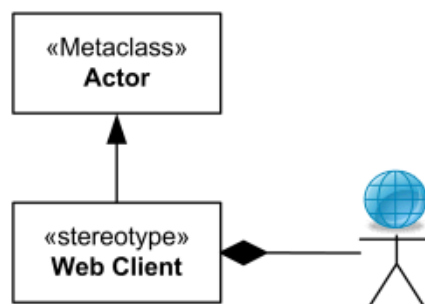
9.3 Syntax of Stereotypes

Stereotypes are written inside: << >>

Example: <<interface>> PaymentService

9.4 Diagram of Stereotypes

Example of UML Stereotypes



9.5 Common Types of Stereotypes

Stereotype	Meaning
<<entity>>	Represents database or business object
<<boundary>>	Represents user interface
<<controller>>	Controls application logic
<<interface>>	Defines interface
<<utility>>	Represents helper/utility class
<<persistent>>	Stores data permanently

9.6 Working of Stereotypes

Step 1: Create UML Element

A class or component is created.

Example:

- Student class
- Login page
- Payment controller

Step 2: Assign Appropriate Stereotype

The role of the element is identified.

Example:

- Student → <<entity>>
- LoginPage → <<boundary>>
- PaymentManager → <<controller>>

Step 3: Use in UML Diagram

The stereotype is displayed above the element name. This helps developers quickly identify the purpose of each component.

9.7 Important Stereotypes in OOSE

9.7.1 <<entity>>

Definition: Represents data-oriented or business classes. These classes usually store information.

Example

- Student
- Employee
- Account

Diagram

```
+-----+
| <<entity>> |
| Employee   |
+-----+
```

Advantages

- Represents business data clearly
 - Easy database mapping
-

9.7.2 <<boundary>>

Definition: Represents interaction between system and user.

Usually used for:

- Forms
- Screens
- User interfaces

Example

- Login Page
- Registration Form

Diagram

```
+-----+
| <<boundary>> |
| LoginPage    |
+-----+
```

Advantages

- Separates UI from business logic
 - Improves system organization
-

9.7.3 <<controller>>

Definition: Controls system flow and business logic. Acts as mediator between boundary and entity classes.

Example

- PaymentController
- UserManager

Diagram

```
+-----+
| <<controller>> |
| PaymentController |
+-----+
```

Advantages

- Better control handling
 - Improves modularity
-

9.7.4 <<interface>>

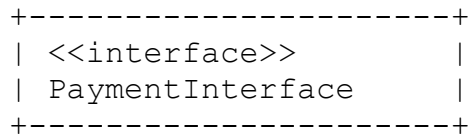
Definition: Represents an interface in object-oriented programming. Defines methods without implementation.

Example

- PaymentInterface

- DatabaseInterface

Diagram



Advantages

- Supports abstraction
- Encourages loose coupling

9.8 Advantages of Stereotypes

- a) **Improves Readability:** Diagrams become easier to understand.
- b) **Adds Extra Information:** Provides additional meaning to UML elements.
- c) **Better Classification:** Separates UI, logic, and data components.
- d) **Simplifies Large Systems:** Helps organize complex software designs.
- e) **Supports Reusability:** Clearly defined roles improve reuse.

9.9 Applications of Stereotypes

- a) **Software Design:** Used in class and component diagrams.

b) MVC Architecture

Used to represent:

- Model
- View
- Controller

- c) **Web Applications:** Defines frontend and backend classes.
- d) **Banking Systems:** Represents account, transaction, and service components.

e) **E-Commerce Applications:** Used for cart, payment, and user modules.

9.10 Stereotypes vs Normal Classes

Feature	Stereotypes	Normal Classes
Purpose	Add extra meaning	Basic class representation
Information	More descriptive	Limited information
UML Extension	Yes	No
Readability	High	Medium
Example	<< Student	Student

9.11 Real Life Example Using Stereotypes

Online Shopping System

Component	Stereotype
Product	<<entity>>
CartPage	<<boundary>>
OrderController	<<controller>>
PaymentInterface	<<interface>>

This classification helps developers understand system architecture easily.

9.12 Conclusion

Stereotypes are important UML extension mechanisms used to provide additional meaning to UML elements.

They help:

- Improve diagram readability
- Organize system components
- Separate user interface, logic, and data
- Simplify complex software systems

Common stereotypes include:

- <<entity>>
- <<boundary>>
- <<controller>>
- <<interface>>

Stereotypes are widely used in Object-Oriented Software Engineering for better system modeling and design.

10. CRC Card

10.1 Definition of CRC Card

CRC Card stands for:

- **C** → Class
- **R** → Responsibilities
- **C** → Collaborators

CRC Card is a simple object-oriented design tool used to identify:

- Classes
- Responsibilities of classes
- Interaction between classes

It helps developers understand how objects work together in a software system.

CRC cards are mainly used during:

- Object Oriented Analysis (OOA)
- Object Oriented Design (OOD)

The concept was introduced by:

- Ward Cunningham
 - Kent Beck
-

10.2 Purpose of CRC Card

CRC Cards are used to:

- Identify system classes
- Assign responsibilities

- Find collaboration between objects
- Simplify software design
- Improve object-oriented thinking

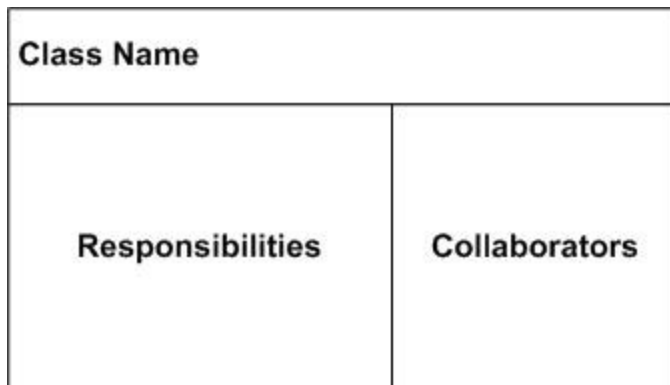
They help developers design systems before coding starts.

10.3 Structure of CRC Card

A CRC card is divided into three sections.

Section	Description
Class	Name of class
Responsibilities	Tasks performed by class
Collaborators	Other classes helping the class

10.4 Diagram of CRC Card



10.5 Working of CRC Card

Step 1: Identify Classes

Classes are identified from system requirements.

Example:

- Student
- Book
- Library
- Account

Step 2: Assign Responsibilities

Responsibilities define what a class should do.

Example:

Student Class Responsibilities

- Borrow book
- Return book
- Search book

Step 3: Identify Collaborators

Collaborators are classes that help complete responsibilities.

Example: Student collaborates with Book class.

Step 4: Analyze Interactions

Developers study how classes interact with each other. This helps improve software design.

10.6 Advantages of CRC Card

- a) **Easy to Understand:** Simple and user-friendly design method.
 - b) **Improves Object-Oriented Thinking:** Helps identify classes and responsibilities clearly.
 - c) **Encourages Team Discussion:** Useful during brainstorming and software planning.
 - d) **Better Class Design:** Helps distribute responsibilities properly.
 - e) **Reduces Design Complexity:** Simplifies large systems into smaller objects.
 - f) **Helps Before Coding:** Errors can be identified early in design phase.
-

10.7 Applications of CRC Card

- a) **Software Design:** Used during object-oriented design phase.
- b) **Banking Systems:** ATM and account management systems.

- c) **E-Commerce Applications:** Cart, order, and payment systems.
 - d) **Library Management Systems:** Book issue and return systems.
 - e) **Hospital Management:** Patient and doctor management systems.
 - f) **Educational Projects:** Widely used for learning OOP concepts.
-

10.8 CRC Card vs Class Diagram

Feature	CRC Card	Class Diagram
Purpose	Identify responsibilities	Show detailed structure
Complexity	Simple	More detailed
Representation	Text-based card	UML graphical diagram
Focus	Collaboration	Relationships and attributes
Usage Phase	Early design	Detailed design

10.9 Important Concepts in CRC Card

i) **Class:** Represents an object or entity.

Example:

- Student
- Employee

ii) **Responsibilities:** Tasks performed by class.

Example:

- Login
- Search data
- Print report

iii) **Collaborators:** Other classes helping in task completion.

Example:

- Database
- Payment system

10.10 Steps to Design CRC Card

1. Study system requirements
2. Identify classes
3. Assign responsibilities
4. Identify collaborators
5. Refine class interactions
6. Validate object behavior

10.11 Real Life Example of CRC Card

College Management System

Class	Responsibilities	Collaborators
Student	Register course, Pay fees	Course, Payment
Teacher	Upload marks, Teach course	Student
Admin	Manage records	Database

10.12 Conclusion

CRC Card is an important object-oriented design technique used to identify:

- Classes
- Responsibilities
- Collaborators

It simplifies software design and improves understanding of object interactions.

CRC cards are useful because they:

- Encourage teamwork
- Improve class design
- Reduce complexity
- Help before implementation

They are widely used in Object-Oriented Software Engineering for analysis and system design.

11. State Chart Diagram

11.1 Definition of State Chart Diagram

A State Chart Diagram is a behavioral UML diagram used to represent the different states of an object and the transitions between those states during its life cycle.

It shows:

- Different states of an object
- Events causing state changes
- Flow from one state to another

State chart diagrams are mainly used to model:

- Dynamic behavior of systems
- Object life cycle
- Event-driven systems

They help developers understand how an object behaves in different situations.

11.2 Purpose of State Chart Diagram

State chart diagrams are used to:

- Represent object behavior
- Show state transitions
- Model real-time systems
- Understand event-driven activities
- Simplify dynamic system analysis

They are useful when object behavior changes over time.

11.3 Basic Components of State Chart Diagram

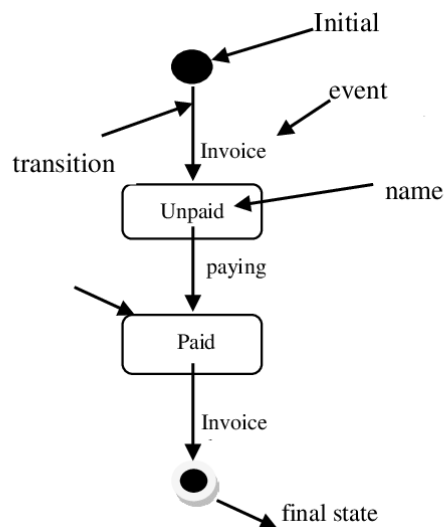
Component	Meaning
State	Condition or situation of object
Transition	Movement from one state to another

Component	Meaning
Event	Action causing transition
Initial State	Starting point
Final State	Ending point

11.4 Symbols Used in State Chart Diagram

Symbol	Purpose
Filled Circle	Initial State
Rounded Rectangle	State
Arrow	Transition
Double Circle	Final State

11.5 Diagram of State Chart Diagram



11.6 Working of State Chart Diagram

Step 1: Initial State

The object starts from an initial state.

Example: ATM machine is idle.

Step 2: Event Occurs

An event triggers state transition.

Example: User inserts ATM card.

Step 3: State Transition

The object changes from one state to another.

Example: Idle → Card Verification

Step 4: Activity Execution

Specific action is performed inside the state.

Example:

- PIN validation
- Transaction processing

Step 5: Final State

After completing all operations, object reaches final state.

Example: Transaction completed

11.7 State Transition Example

Current State	Event	Next State
Idle	Insert Card	Verification
Verification	Valid PIN	Transaction
Transaction	Withdraw Cash	Receipt
Receipt	Finish	Final State

11.8 Advantages of State Chart Diagram

a) **Represents Dynamic Behavior:** Shows how object changes with time.

- b) Easy Understanding of Object Life Cycle:** Clearly explains states and transitions.
 - c) Useful for Real-Time Systems:** Best for systems reacting to events.
 - d) Improves System Analysis:** Helps detect missing states or transitions.
 - e) Simplifies Complex Logic:** Breaks dynamic behavior into manageable states.
 - f) Better Software Design:** Improves object-oriented modeling.
-

11.9 Applications of State Chart Diagram

- a) ATM Systems:** Transaction processing states.
 - b) Traffic Light Systems:** Signal state changes.
 - c) Online Shopping Systems:** Order processing workflow.
 - d) Mobile Applications:** Screen navigation and user interaction.
 - e) Elevator Control Systems:** Movement and operation states.
 - f) Network Communication Systems:** Connection establishment and termination.
-

11.10 State Chart Diagram vs Activity Diagram

Feature	State Chart Diagram	Activity Diagram
Focus	Object states	Workflow/process
Purpose	Dynamic behavior	Process flow
Main Element	States	Activities
Transition Trigger	Events	Activity flow
Usage	Object life cycle	Business process

11.11 Important Concepts in State Chart Diagram

- i) State:** Condition of an object at a particular time.

Example:

- Idle
- Processing
- Completed

ii) Event: Action causing state change.

Example:

- Button click
- Payment success

iii) Transition: Movement from one state to another.

Example: Login → Dashboard

iv) Initial State: Starting point of process.

v) Final State: Ending point of process.

11.12 Conclusion

State Chart Diagram is an important UML behavioral diagram used to represent:

- States of objects
- State transitions
- Dynamic behavior of systems

It helps developers:

- Understand object life cycle
- Design event-driven systems
- Simplify dynamic processes
- Improve software modeling

State chart diagrams are widely used in real-time systems, banking systems, automation systems, and object-oriented software engineering.

12. Component Diagram vs Class Diagram

12.1 Definition of Component Diagram

A **Component Diagram** is a UML structural diagram that shows the physical components of a software system and the relationship between them.

It represents:

- Software modules
- Libraries
- Executable files
- Packages
- Interfaces
- Dependencies between components

It mainly focuses on the **implementation structure** of the system.

Examples:

- Payment module
 - Login module
 - Database module
 - Reporting service
-

12.2 Definition of Class Diagram

A **Class Diagram** is a UML structural diagram that represents the classes, objects, attributes, methods, and relationships in a system.

It mainly focuses on:

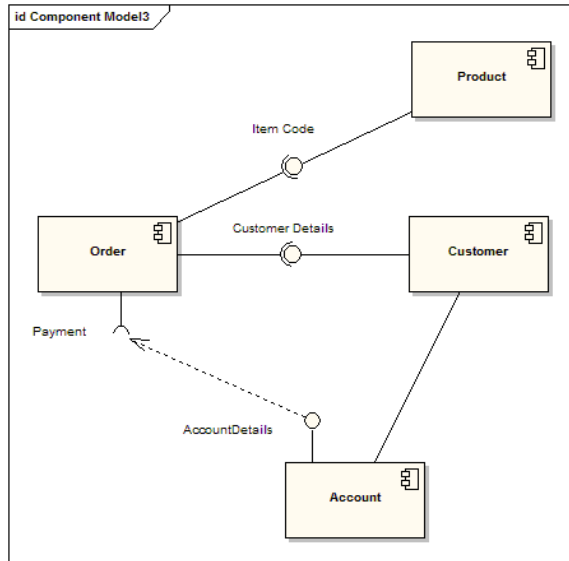
- Object structure
- Data members
- Functions
- Relationships among classes

It is used during **Object Oriented Design (OOD)**.

Examples:

- Student class
 - Employee class
 - BankAccount class
-

12.3 Diagram of Component Diagram



Explanation

- Each box represents a software component.
 - Arrows show dependency or communication.
 - Components work together to build the complete application.
-

12.4 Working of Component Diagram

The Component Diagram works in the following way:

- System Division:** The software system is divided into independent modules called components.
- Interface Communication:** Components communicate through interfaces.
- Dependency Management:** One component may depend on another component for services.
- Deployment Support:** The diagram helps developers understand how software modules are connected during implementation.

Example: Login component communicates with database component for authentication.

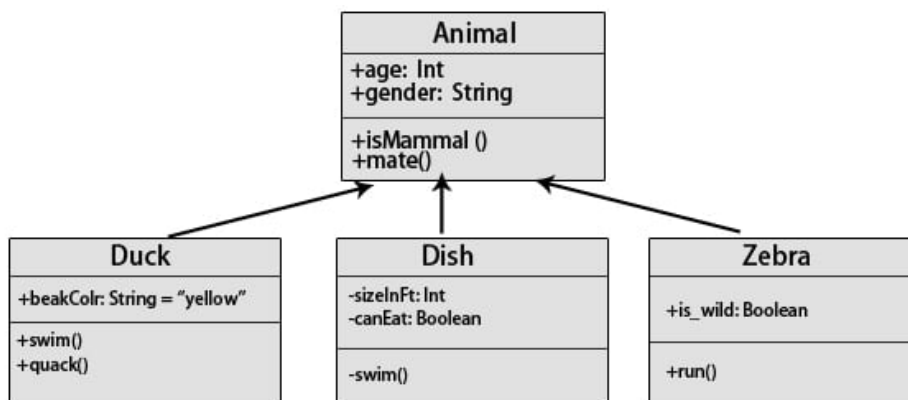
12.5 Advantages of Component Diagram

- a) **Easy System Understanding:** Helps understand software architecture easily.
 - b) **Better Modularity:** Divides system into reusable modules.
 - c) **Simplifies Maintenance:** Changes in one component affect fewer parts.
 - d) **Reusability:** Components can be reused in other projects.
 - e) **Helps Deployment:** Useful during software implementation and deployment.
-

12.6 Applications of Component Diagram

- a) **Enterprise Applications:** Used in banking and ERP systems.
 - b) **Web Applications:** Represents frontend, backend, and database modules.
 - c) **Distributed Systems:** Shows communication between services.
 - d) **Microservices Architecture:** Represents independent services.
 - e) **Large Software Projects:** Used for system organization and maintenance.
-

12.7 Diagram of Class Diagram



Explanation

- Upper section → Class name
- Middle section → Attributes

- Lower section → Methods
 - Arrows show relationships between classes
-

12.8 Working of Class Diagram

The Class Diagram works as follows:

- i) **Class Identification:** Classes of the system are identified.
- ii) **Attribute Definition:** Data members of classes are specified.
- iii) **Method Definition:** Functions performed by classes are defined.
- iv) **Relationship Creation:** Relationships like association, inheritance, and aggregation are shown.

Example: A Student class can register for a Course class.

12.9 Advantages of Class Diagram

- a) **Clear Structure:** Provides clear understanding of system design.
 - b) **Better Coding:** Acts as blueprint for coding.
 - c) **Reusability:** Classes can be reused.
 - d) **Easy Maintenance:** Makes modifications easier.
 - e) **Supports Object Oriented Design:** Represents OOP concepts effectively.
-

12.10 Applications of Class Diagram

- a) **Software Design:** Used before coding.
- b) **Database Design:** Helps design database tables.
- c) **Educational Systems:** Used in student management systems.
- d) **Banking Systems:** Represents account and customer classes.

e) **E-Commerce Applications:** Used for products, orders, customers, etc.

12.11 Difference Between Component Diagram and Class Diagram

Basis	Component Diagram	Class Diagram
Purpose	Shows software modules/components	Shows classes and objects
Focus	Physical implementation	Logical design
Used In	System architecture	Object-oriented design
Represents	Components and interfaces	Classes, attributes, methods
Level	High-level view	Detailed view
Relationships	Dependencies between modules	Relationships between classes
Main Usage	Deployment and implementation	System modeling and coding
Example	Login module, Payment service	Student class, Employee class

12.12 Conclusion

Component Diagram and Class Diagram are important UML structural diagrams used in software engineering.

- **Class Diagram** focuses on the internal structure of classes and object-oriented modeling.
- **Component Diagram** focuses on software modules and system implementation architecture.

Both diagrams help developers design, understand, maintain, and develop large software systems efficiently.

13. Deployment Diagram

13.1 Definition of Deployment Diagram

A **Deployment Diagram** is a UML structural diagram that represents the physical deployment of hardware and software components in a system.

It shows:

- Hardware devices (nodes)
- Software components
- Network connections
- Communication between systems

Deployment diagrams are mainly used during the **implementation and deployment phase** of software development.

Examples:

- Web application deployment
 - Client-server systems
 - Cloud-based applications
 - ATM systems
-

13.2 Purpose of Deployment Diagram

The main purposes are:

- To represent physical architecture of the system
 - To show where software components are installed
 - To describe communication between hardware devices
 - To help developers understand system deployment
-

13.3 Basic Components of Deployment Diagram

Component	Description
Node	Physical hardware device
Artifact	Software component or executable file
Communication Path	Connection between nodes
Deployment	Installation of software on hardware

13.4 Types of Nodes

i) Device Node: Represents physical hardware devices.

Examples:

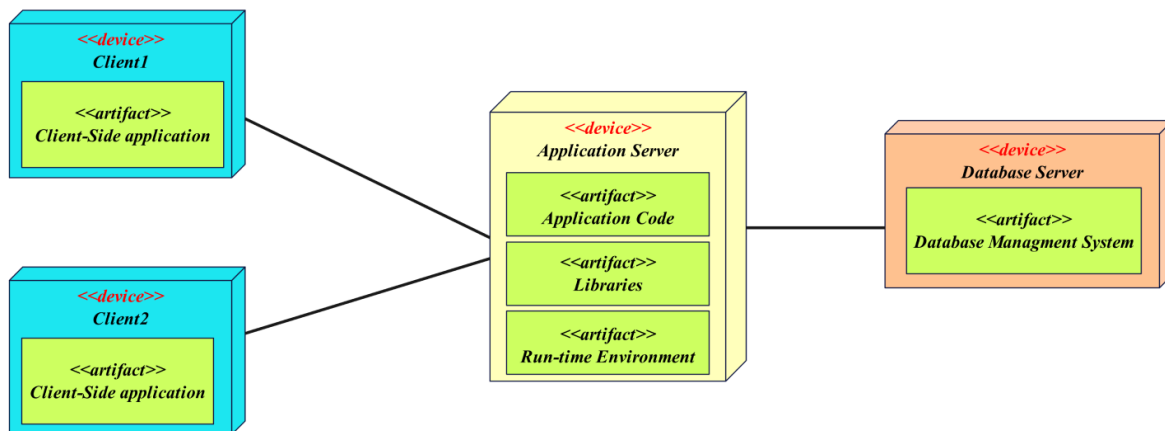
- Computer
- Mobile phone
- Server
- Printer

ii) Execution Environment Node: Represents software environments where programs run.

Examples:

- JVM
 - Operating system
 - Web server
-

13.5 Diagram of Deployment Diagram



13.6 Working of Deployment Diagram

The Deployment Diagram works as follows:

i) Client Request: The client device sends a request to the server.

Example: User opens shopping website.

ii) Server Processing: The web server processes user requests.

Functions:

- Authentication
- Business logic
- Data processing

iii) Database Communication: The server communicates with the database server.

Functions:

- Store data
- Retrieve records
- Update information

iv) Response Generation: The processed response is sent back to the client.

Example: Product details displayed to user.

13.7 Advantages of Deployment Diagram

- a) Clear Physical Architecture:** Shows hardware and software structure clearly.
 - b) Better System Understanding:** Helps understand communication between devices.
 - c) Simplifies Deployment:** Useful during software installation.
 - d) Improves Maintenance:** Makes troubleshooting easier.
 - e) Supports Distributed Systems:** Useful for network-based applications.
-

13.8 Applications of Deployment Diagram

- a) Web Applications:** Used in online shopping and banking systems.
- b) Client-Server Systems:** Represents interaction between clients and servers.
- c) Cloud Computing:** Shows cloud deployment architecture.
- d) Mobile Applications:** Represents mobile-server communication.

e) **Distributed Systems:** Used in large-scale enterprise applications.

13.9 Deployment Diagram vs Component Diagram

Basis	Deployment Diagram	Component Diagram
Purpose	Shows physical deployment	Shows software components
Focus	Hardware + software mapping	Software architecture
Represents	Nodes and communication	Components and interfaces
Usage	Implementation phase	Design phase
Main Elements	Nodes, artifacts	Components, dependencies
Example	Client-server setup	Login module, payment module

13.10 Real Life Example of Deployment Diagram

ATM System

Components Used:

- ATM machine
- Bank server
- Database server
- Network connection

Working:

1. User inserts ATM card.
2. ATM sends request to bank server.
3. Bank server checks database.
4. Transaction result is returned to ATM.

Deployment diagram helps represent all these physical connections.

13.11 Conclusion

A Deployment Diagram is an important UML diagram used to represent the physical arrangement of hardware devices and software components in a system.

It helps developers understand:

- System deployment
- Network communication
- Hardware configuration
- Software installation

Deployment diagrams are widely used in web applications, distributed systems, cloud platforms, and enterprise software systems.

14. Design Patterns

14.1 Definition of Design Pattern

A **Design Pattern** is a reusable solution to commonly occurring problems in software design.

It provides a standard way to solve software development problems using Object Oriented principles.

Design patterns are not complete programs.

They are templates or guidelines that help developers write better, reusable, and maintainable code.

Design patterns improve:

- Software structure
 - Reusability
 - Flexibility
 - Maintainability
-

14.2 Need of Design Patterns

During software development, developers face many common problems such as:

- Code duplication
- Tight coupling
- Complex object creation

- Difficult maintenance

Design patterns help solve these problems using proven solutions.

14.3 Features of Design Patterns

- i) **Reusability:** Patterns can be reused in many projects.
 - ii) **Flexibility:** Software becomes easier to modify.
 - iii) **Maintainability:** Code becomes easier to maintain.
 - iv) **Scalability:** Applications can grow easily.
 - v) **Standardization:** Provides standard software design methods.
-

14.4 Categories of Design Patterns

Design patterns are mainly divided into three categories:

Category	Purpose
Creational Patterns	Object creation
Structural Patterns	Class and object structure
Behavioral Patterns	Communication between objects

14.5 Creational Design Patterns

Creational patterns deal with object creation mechanisms.

Examples:

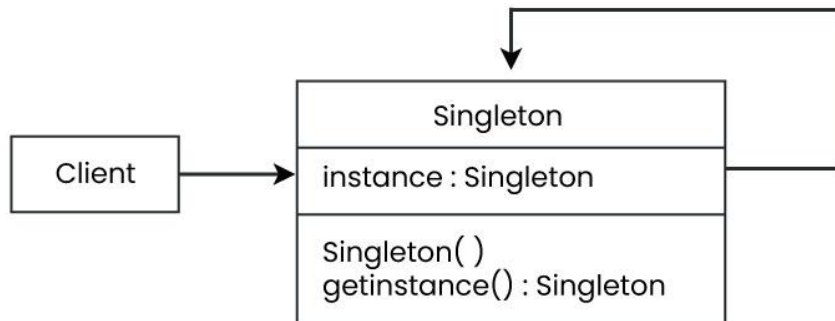
- Singleton
- Factory Method
- Builder
- Prototype

Example: Singleton Pattern

Definition: Singleton pattern ensures that only one object of a class is created.

Example: Database connection manager.

Diagram



Advantages

- Saves memory
- Controlled object creation

Applications

- Database connection
- Logger system
- Configuration manager

14.6 Structural Design Patterns

Structural patterns define relationships between classes and objects.

Examples:

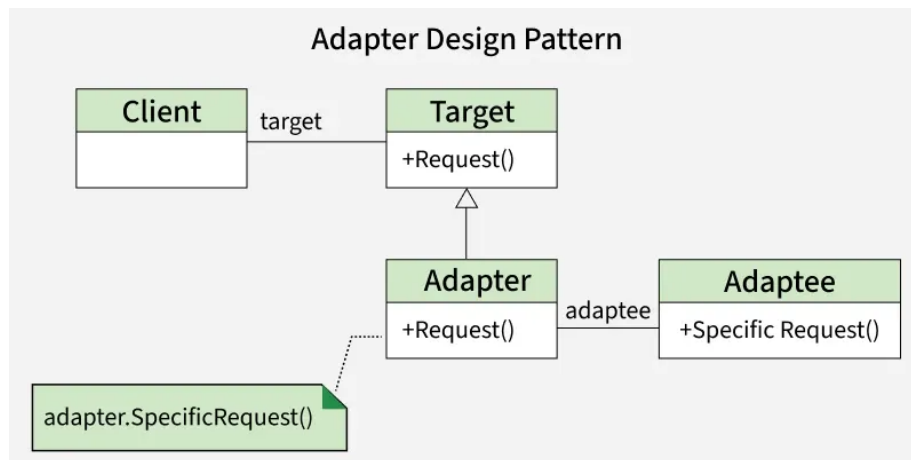
- Adapter
- Bridge
- Composite
- Decorator

Example: Adapter Pattern

Definition: Adapter pattern connects incompatible interfaces.

Example: Mobile charger adapter.

Diagram



Advantages

- Improves compatibility
- Reuses existing code

Applications

- Legacy system integration
- API integration

14.7 Behavioral Design Patterns

Behavioral patterns manage communication between objects.

Examples:

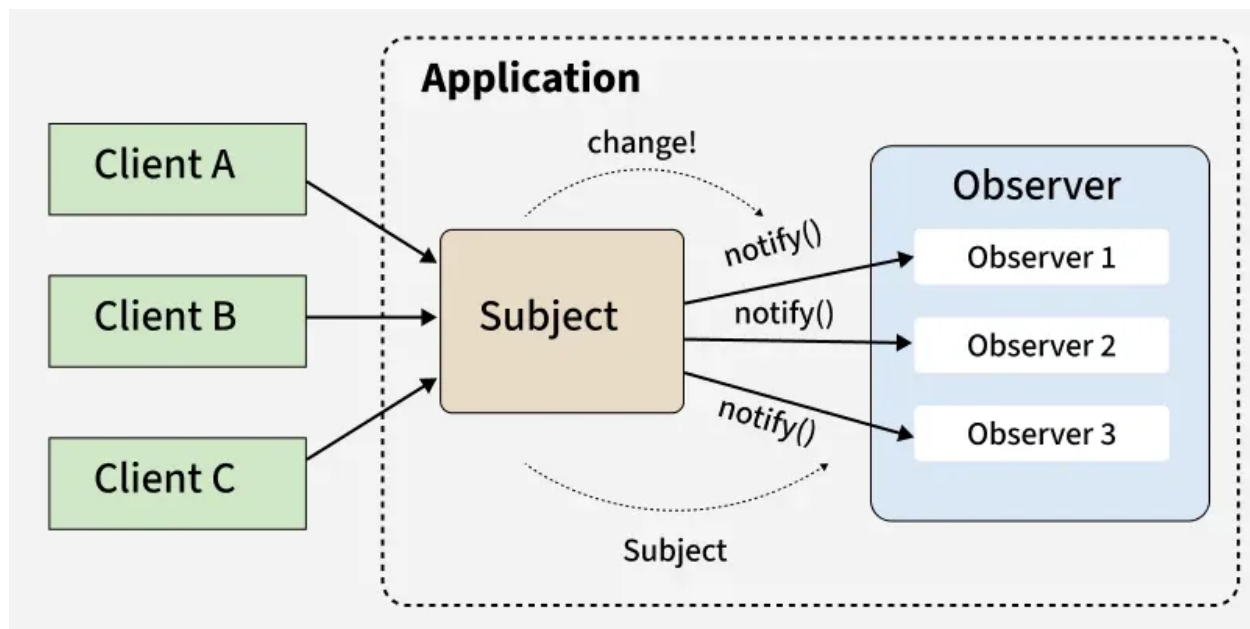
- Observer
- Strategy
- Command
- State

Example: Observer Pattern

Definition: Observer pattern allows one object to notify multiple dependent objects automatically.

Example: YouTube subscriber notification system.

Diagram



Advantages

- Loose coupling
- Automatic updates

Applications

- Notification systems
- Event management systems

14.8 Working of Design Patterns

The working process of design patterns is:

i) Problem Identification: A common software design problem is identified.

ii) Pattern Selection: Suitable design pattern is selected.

Example:

- Singleton for single object creation
- Observer for notification systems

iii) Implementation: Pattern structure is implemented in code.

iv) Reusability: The same pattern can be reused in future projects.

14.9 Advantages of Design Patterns

a) Code Reusability: Same solutions can be reused.

b) Better Maintainability: Code becomes organized and easier to manage.

c) Reduced Complexity: Simplifies software structure.

d) Faster Development: Developers use ready solutions.

e) Improved Communication: Developers understand standard patterns easily.

14.10 Applications of Design Patterns

a) Enterprise Applications: Used in banking and ERP systems.

b) Web Development: Used in MVC architecture and frameworks.

c) Mobile Applications: Improves app structure and maintenance.

d) Game Development: Used for object management.

e) Software Frameworks: Java Spring and Hibernate use design patterns.

14.11 Design Patterns vs Object Oriented Modeling

Basis	Design Patterns	Object Oriented Modeling
Purpose	Solves common design problems	Represents system structure

Basis	Design Patterns	Object Oriented Modeling
Focus	Reusable solutions	System analysis and design
Usage	During coding/design	During modeling phase
Nature	Predefined templates	Graphical representation
Example	Singleton, Observer	UML diagrams

14.12 Real Life Examples of Design Patterns

Design Pattern	Real Life Example
Singleton	President of a country
Factory	Car manufacturing factory
Observer	YouTube notifications
Adapter	Mobile charger adapter
Builder	House construction process

14.13 Conclusion

Design Patterns are reusable software design solutions that help developers build efficient, flexible, and maintainable applications.

They improve:

- Code quality
- Reusability
- Software architecture
- Development speed

Design patterns are widely used in modern software systems, frameworks, web applications, and enterprise applications.

15. OOD v/s OOA

15.1 Definition of OOA (Object Oriented Analysis)

Object Oriented Analysis (OOA) is the process of analyzing system requirements using object-oriented concepts.

It focuses on:

- Understanding user requirements
- Identifying objects
- Identifying classes and relationships
- Studying system behavior

OOA answers:

- **What should the system do?**

Examples:

- Identifying users in ATM system
 - Finding objects like Account, Customer, Transaction
-

15.2 Definition of OOD (Object Oriented Design)

Object Oriented Design (OOD) is the process of designing the system after analysis.

It focuses on:

- Designing classes
- Defining methods
- Creating relationships
- Building software structure

OOD answers:

- **How will the system work?**

Examples:

- Designing Account class
 - Creating methods like withdraw(), deposit()
-

15.3 Objectives of OOA

Main objectives are:

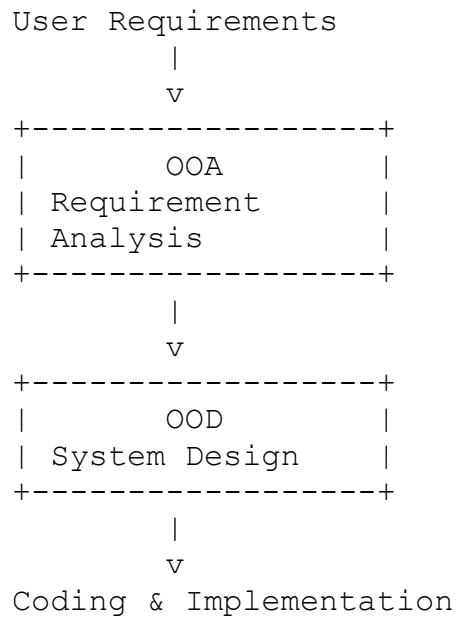
- Understand system requirements
 - Identify objects and classes
 - Define system functionality
 - Create analysis models
 - Reduce requirement confusion
-

15.4 Objectives of OOD

Main objectives are:

- Design software architecture
 - Convert analysis into implementation
 - Improve maintainability
 - Create reusable classes
 - Prepare for coding
-

15.5 Diagram of OOA and OOD Process



15.6 Working of OOA

OOA works in the following steps:

i) Requirement Gathering: System requirements are collected from users.

Example: ATM should allow cash withdrawal and balance checking.

ii) Object Identification: Important objects are identified.

Examples:

- Customer
- ATM
- Bank Account

iii) Relationship Identification: Relationships between objects are identified.

Example: Customer owns Account

iv) Analysis Modeling: Use case diagrams and analysis models are created.

15.7 Working of OOD

OOD works in the following steps:

i) Class Design: Classes are designed from analysis objects.

Example:

- Account class
- Customer class

ii) Attribute Definition: Data members are added.

Example:

- accountNumber
- balance

iii) Method Definition: Functions are created.

Example:

- deposit()
- withdraw()

iv) Relationship Design: Inheritance, association, aggregation are designed.

v) Interface Design: Communication between classes is defined.

15.8 Advantages of OOA

a) Better Requirement Understanding: Helps understand user needs clearly.

b) Reduces Errors: Detects requirement problems early.

c) Improves Communication: Developers and users understand system easily.

d) Better Planning: Provides strong base for design phase.

15.9 Advantages of OOD

a) Code Reusability: Classes can be reused.

b) Better Maintainability: Software becomes easy to maintain.

c) Scalability: New features can be added easily.

d) Flexibility: Design changes become easier.

e) Supports Object Oriented Programming: Implements OOP concepts effectively.

15.10 Applications of OOA and OOD

a) Banking Systems: ATM and online banking systems.

b) E-Commerce Applications: Shopping and payment systems.

c) Hospital Management Systems: Patient and doctor management.

d) Educational Systems: Course registration and student systems.

e) **Enterprise Applications:** ERP and CRM systems.

15.11 Difference Between OOA and OOD

Basis	OOA	OOD
Full Form	Object Oriented Analysis	Object Oriented Design
Purpose	Analyze requirements	Design the solution
Focus	What system should do	How system will work
Stage	Early phase	Design phase
Main Activity	Requirement analysis	System design
Output	Analysis model	Design model
Concern	Problem understanding	Solution creation
Diagrams Used	Use case, activity diagram	Class, sequence diagram
User Involvement	High	Medium
Coding	No coding focus	Prepares for coding

15.12 Real Life Example

ATM System

In OOA:

Objects identified are:

- Customer
- ATM
- Account
- Transaction

System requirements:

- Withdraw money
- Deposit money
- Check balance

In OOD:

Classes are designed:

- Customer class
- Account class
- ATM class

Methods created:

- withdraw()
 - deposit()
 - checkBalance()
-

15.13 Conclusion

OOA and OOD are important phases of Object Oriented Software Engineering.

- **OOA** focuses on understanding and analyzing system requirements.
- **OOD** focuses on designing software structure and implementation.

Both help in developing:

- Reliable software
 - Reusable systems
 - Maintainable applications
 - Efficient object-oriented solutions
-

16. Test Plan

16.1 Definition of Test Plan

A **Test Plan** is a formal document that describes the testing strategy, objectives, resources, schedule, and activities required for testing a software application.

It acts as a blueprint for the testing process.

A test plan defines:

- What will be tested
- How testing will be done
- When testing will be performed
- Who will perform testing

It helps ensure that the software works correctly and meets user requirements.

16.2 Objectives of Test Plan

Main objectives are:

- Ensure software quality
 - Detect errors and bugs
 - Define testing process clearly
 - Reduce testing confusion
 - Improve software reliability
 - Save time and cost
-

16.3 Need of Test Plan

Without a proper test plan:

- Testing becomes unorganized
- Important features may be missed
- Errors may remain undetected
- Project delays may occur

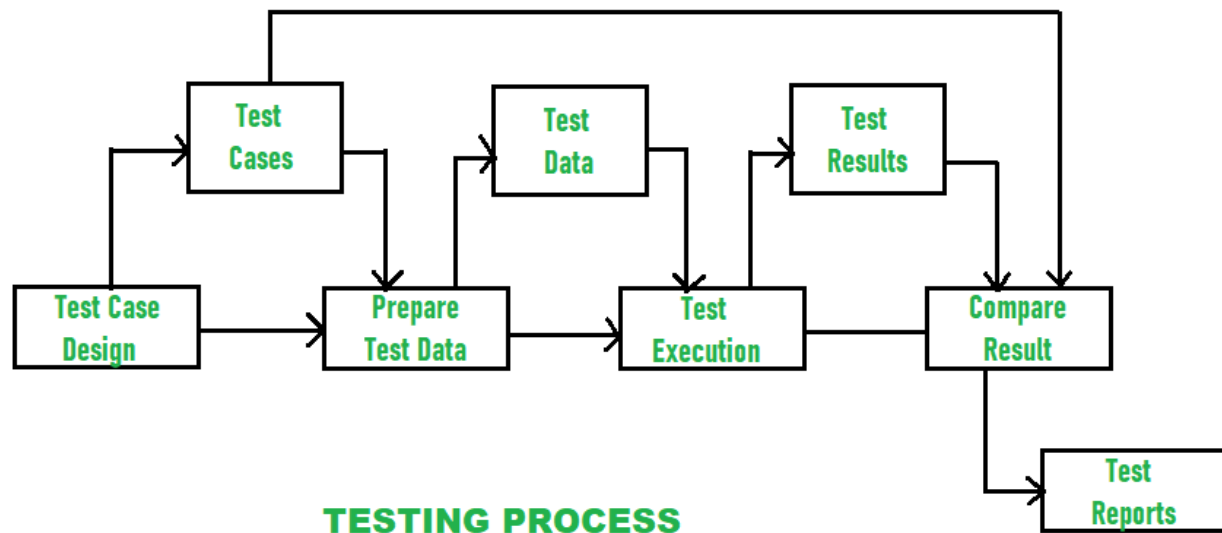
A test plan helps perform systematic and controlled testing.

16.4 Components of Test Plan

Component	Description
Test Objective	Purpose of testing
Scope	Features to be tested
Test Strategy	Testing approach

Component	Description
Test Cases	Individual test conditions
Resources	Tools and team members
Schedule	Testing timeline
Risk Analysis	Possible testing risks
Deliverables	Reports and documents

16.5 Diagram of Test Plan Process



16.6 Working of Test Plan

The Test Plan works in the following steps:

i) Requirement Analysis: Testing team studies software requirements.

Example:

- Login functionality
- Payment system
- Registration process

ii) Test Objective Definition: Testing goals are identified.

Example:

- Verify correct login functionality
- Check payment security

iii) Test Strategy Selection: Suitable testing methods are selected.

Examples:

- Unit testing
- Integration testing
- System testing

iv) Test Case Preparation: Different test cases are created.

Example:

- Valid login test
- Invalid password test

v) Resource Allocation: Testing tools and team members are assigned.

vi) Test Execution: Testing is performed on software.

vii) Bug Reporting: Detected bugs are documented and fixed.

viii) Final Test Report: Final testing status is prepared.

16.7 Types of Testing Used in Test Plan

Testing Type	Purpose
Unit Testing	Tests individual modules
Integration Testing	Tests module interaction
System Testing	Tests complete system
Acceptance Testing	Verifies user requirements

Testing Type	Purpose
Regression Testing	Checks old functionality after changes

16.8 Advantages of Test Plan

- a) **Organized Testing:** Provides proper testing structure.
 - b) **Better Quality:** Improves software reliability.
 - c) **Time Management:** Helps complete testing on schedule.
 - d) **Risk Reduction:** Reduces chances of software failure.
 - e) **Better Communication:** Improves coordination among team members.
 - f) **Cost Saving:** Detects issues early and reduces maintenance cost.
-

16.9 Applications of Test Plan

- a) **Web Applications:** Used in e-commerce and banking systems.
 - b) **Mobile Applications:** Testing Android and iOS applications.
 - c) **Enterprise Software:** ERP and CRM system testing.
 - d) **Healthcare Systems:** Testing hospital management software.
 - e) **Embedded Systems:** Used in IoT and hardware-based applications.
-

16.10 Guidelines for Developing Test Plan

- i) **Understand Requirements Clearly:** Requirements should be properly analyzed.
- ii) **Define Testing Scope:** Specify what will and will not be tested.
- iii) **Choose Proper Testing Methods:** Use suitable testing techniques.
- iv) **Create Realistic Schedule:** Testing deadlines should be achievable.

v) Prepare Backup Plans: Handle risks and unexpected problems.

vi) Maintain Proper Documentation: All testing activities should be documented.

16.11 Test Plan vs Test Case

Basis	Test Plan	Test Case
Purpose	Overall testing strategy	Tests one specific condition
Scope	Complete testing process	Individual functionality
Prepared By	Test manager	Test engineer
Level	High level	Detailed level
Contains	Objectives, schedule, resources	Input, output, steps
Example	Complete login module testing	Test valid password

16.12 Real Life Example

Online Shopping Website

Test Plan Includes:

- Login testing
- Product search testing
- Payment gateway testing
- Cart functionality testing

Example Test Case:

Input: Correct username and password

Expected Output: User successfully logs in

16.13 Conclusion

A Test Plan is an important document in software testing that defines the complete testing process.

It helps:

- Improve software quality
- Detect errors early
- Manage testing activities
- Ensure reliable software delivery

Test plans are widely used in web applications, enterprise systems, mobile apps, banking software, and large-scale software projects.

17. OO Testing Strategies

17.1 Definition of Object Oriented Testing

Object Oriented Testing (OO Testing) is the process of testing software developed using Object Oriented Programming (OOP) concepts such as:

- Classes
- Objects
- Inheritance
- Polymorphism
- Encapsulation

The main purpose of OO testing is to verify that all classes, objects, and interactions work correctly.

It checks:

- Correctness of classes
 - Communication between objects
 - Behavior of methods
 - System functionality
-

17.2 Need of OO Testing

Object oriented systems are more complex because of:

- Object interaction
- Inheritance
- Dynamic binding
- Reusability

Testing is necessary to:

- Detect bugs
 - Improve reliability
 - Ensure correct behavior
 - Validate object communication
-

17.3 Features of OO Testing

- i) **Class-Based Testing:** Each class is tested individually.
 - ii) **Object Interaction Testing:** Communication between objects is tested.
 - iii) **Reusability Testing:** Reusable components are verified.
 - iv) **Inheritance Testing:** Parent-child class behavior is checked.
 - v) **Encapsulation Testing:** Private data protection is verified.
-

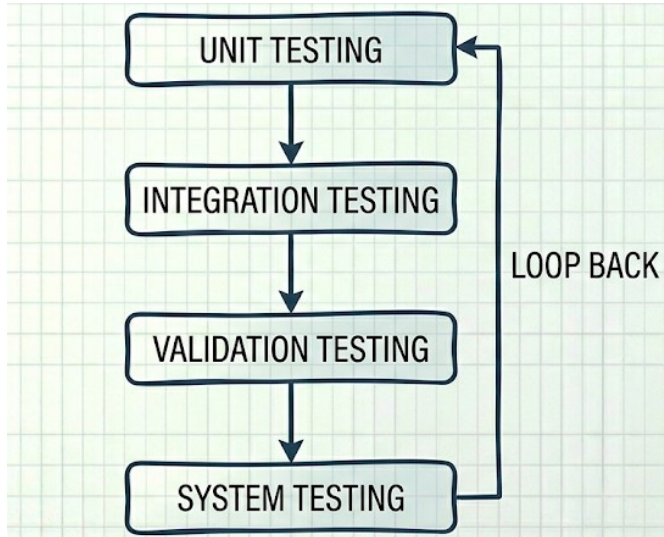
17.4 OO Testing Strategy

OO testing follows different testing strategies to ensure software quality.

Main strategies are:

1. Unit Testing
 2. Integration Testing
 3. Validation Testing
 4. System Testing
-

17.5 Diagram of OO Testing Strategy



17.6 Unit Testing in OO Systems

Definition: Unit testing checks individual classes and methods separately.

Purpose

- Verify class functionality
- Check method correctness

Example

Testing:

- deposit()
- withdraw()
- login()

Advantages

- Detects errors early
 - Simplifies debugging
-

17.7 Integration Testing in OO Systems

Definition: Integration testing checks communication between multiple classes and objects.

Purpose

- Verify object interaction
- Check data flow

Example

Testing communication between:

- Customer class
- Account class
- Transaction class

Types

- Thread-based testing
 - Use-based testing
 - Cluster testing
-

17.8 Validation Testing

Definition: Validation testing checks whether the software meets user requirements.

Purpose

- Ensure correct functionality
- Validate business requirements

Example: Checking whether ATM system performs:

- Cash withdrawal
 - Balance inquiry
 - Fund transfer
-

17.9 System Testing

Definition: System testing verifies the complete integrated system.

Purpose

- Test overall system performance
- Verify complete application

Example: Testing complete banking software.

Checks

- Security
 - Performance
 - Reliability
 - User interface
-

17.10 Common Test Cases in OO Testing

Test Case	Purpose
Method Testing	Check function output
State Testing	Verify object states
Inheritance Testing	Check parent-child behavior
Interface Testing	Verify communication
Exception Testing	Check error handling
Performance Testing	Verify system speed

17.11 Working of OO Testing

The working process is:

- Class Identification:** Classes and objects are identified.
- Test Case Preparation:** Test cases are prepared for methods and interactions.
- Unit Testing:** Each class is tested individually.
- Integration Testing:** Communication among objects is tested.
- Validation and System Testing:** Complete software is tested according to requirements.
- Bug Fixing:** Detected errors are corrected.

17.12 Advantages of OO Testing

- a) **Early Error Detection:** Bugs are identified early.
 - b) **Better Software Quality:** Improves reliability and correctness.
 - c) **Supports Reusability:** Reusable classes can be tested properly.
 - d) **Easier Maintenance:** Helps maintain object-oriented systems.
 - e) **Improved Security:** Checks secure object interaction.
-

17.13 Applications of OO Testing

- a) **Banking Systems:** ATM and online banking applications.
 - b) **E-Commerce Systems:** Shopping and payment applications.
 - c) **Healthcare Systems:** Patient and hospital management systems.
 - d) **Mobile Applications:** Android and iOS apps.
 - e) **Enterprise Applications:** ERP and CRM software.
-

17.14 OO Testing vs Conventional Testing

Basis	OO Testing	Conventional Testing
Focus	Classes and objects	Functions and modules
Based On	OOP concepts	Procedural programming
Complexity	More complex	Less complex
Reusability	High	Low
Testing Unit	Class/object	Function/module

Basis	OO Testing	Conventional Testing
Communication	Object interaction	Function calls

17.15 Real Life Example

ATM System Testing

Unit Testing

Testing:

- withdraw()
- deposit()

Integration Testing

Checking communication between:

- ATM
- Bank server
- Account database

Validation Testing: Verifying user requirements.

System Testing: Testing complete ATM application.

17.16 Conclusion

Object Oriented Testing is an important software testing approach used for object-oriented applications.

It ensures:

- Correct class behavior
- Proper object interaction
- Better software quality
- Reliable performance

OO testing strategies like unit testing, integration testing, validation testing, and system testing help develop efficient and error-free software systems.